

Interactive session
It's time to go hands-on!



Hand-on documentation



Download this presentation

<https://tinyurl.com/3j7cbjh3>



Links:

Openshift Console:

<https://console-openshift-console.apps.cluster-826x2.826x2.sandbox5464.opentlc.com/>

Users:

name: user2..user32
Password: edb-workshop

Devspaces url:

<https://devspaces.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>

Short url to Devspaces:

<https://tinyurl.com/mrxs95pb>

Minio:

UI: <https://minio-ui-minio.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>
API: <https://minio-api-minio.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>

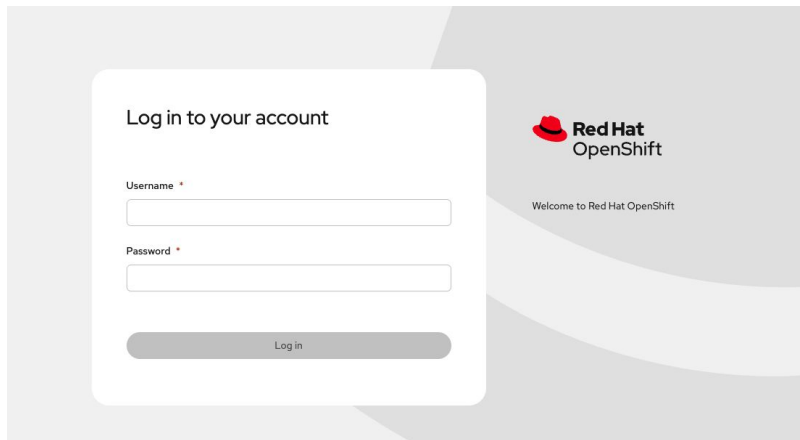
User: minio
Password: edb-workshop



Call to action: Open the Openshift Console

Open the following URL in your browser:

<https://tinyurl.com/yy9dswmk>



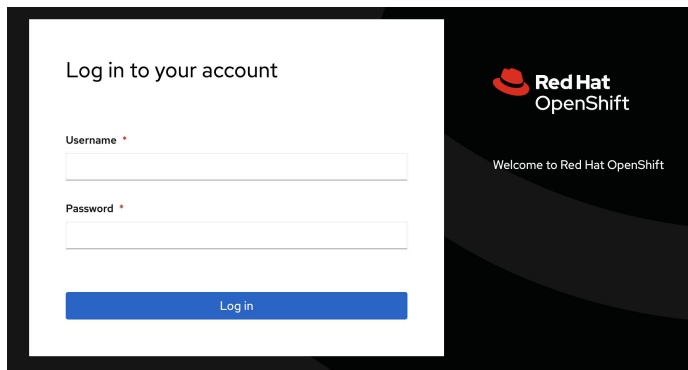
Username and password provided to you

Call to action: Open the DevSpace

Open the following URL in your browser:

<https://red.ht/edb-cph25>

<https://tinyurl.com/yy9dswmk>

A login form with a white background and a black border. The title "Log in to your account" is at the top. Below it are two input fields: "Username" and "Password", each with a red asterisk indicating a required field. At the bottom is a blue "Log in" button. To the right of the form, on a black background, is the Red Hat OpenShift logo and the text "Welcome to Red Hat OpenShift".

Username and password provided to you

Call to action: Open the DevSpace

Authorize Access

openshift-operators-client is requesting permission to access your account (user)

Requested permissions

☒ **user:full**

Full read/write access with all of your permissions

Includes any access you have to escalating resources like secrets

You will be redirected to <https://devspaces.apps.cluster-52d72.dynamic.redhatworkshops.io/oauth/callback>

Allow selected permissions

Deny

Press "Allow selected permissions"

Select a Sample

Select a sample to create your first workspace.

workshop

x

1 item



Postgres on OpenShift
Workshop

CloudNativePG on OpenShift
Workshop

In the Select a Sample section search for
"Workshop" and click on the tile

Call to action: Open the DevSpace

Starting workspace enterisedb-workshop

Progress Logs Events

- ✓ 1 Initializing
- ✓ 2 Checking for the limit of running workspaces
- ✓ 3 Creating a workspace
- 🔄 4 **Waiting for workspace to start**
- 5 Open IDE

Your workshop is loading ...

Get Started with VS Code for the Web

Customize your editor, learn the basics, and start coding

Choose your theme

The right theme helps you focus on your code, is easy on your eyes, and is simply more fun to use.

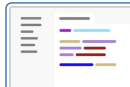
[Browse Color Themes](#)

Tip: Use keyboard shortcut `Ctrl+K Ctrl+T`

- ☐ Just the right amount of UI
- ☐ Rich support for all your languages



Dark Modern



Light Modern



Dark High Contrast



Light High Contrast

[See More Themes...](#)

Select your theme



Do you trust the authors of the files in this workspace?

VS Code - Open Source provides features that may automatically execute files in this workspace.

If you don't trust the authors of these files, we recommend to continue in restricted mode as the files may be malicious. See [our docs](#) to learn more.

/projects (Workspace)

No, I don't trust the authors

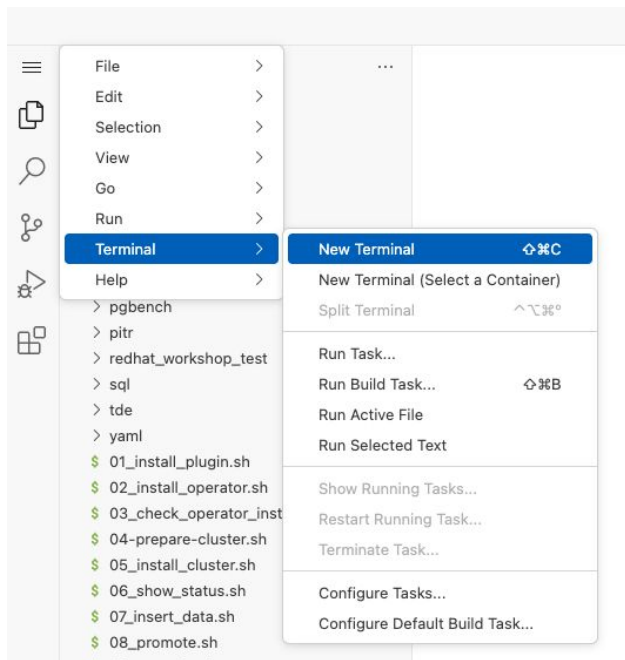
Browse workspace in restricted mode

Yes, I trust the authors

Trust workspace and enable all features

And trust the authors

Call to action: Open the new terminal window



Open two terminal windows

Use case

The environment



Features shown during the demo

- Kubernetes plugin install
- Check the CloudNativePG operator status
- Postgres cluster install
- Insert data in the cluster
- Failover
- Backup
- Recovery
- Scale out/down
- Fencing
- Hibernation
- Monitoring
- Rolling updates (minor and major)

Deployment

High Availability

Administration

Monitoring

Backup and
Recovery

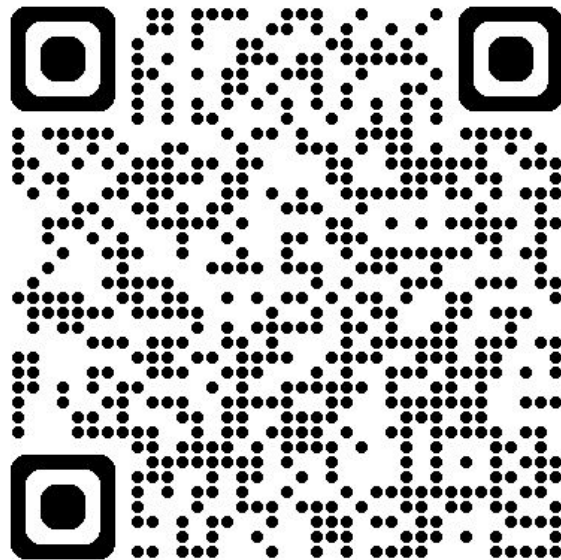
Last CloudNativePG tested version is 1.25



This demo is in 

<https://github.com/sergioenterprisedb/edb-postgres-for-kubernetes-in-openshift>

<http://bit.ly/4duKxm7>



Use case Plug-in installation



The “cnp” plugin for kubectl

- The official CLI for CloudNativePG
 - Available also as RPM or Deb package
- Extends the ‘kubectl’ command:
 - Customize the installation of the operator
 - Status of a cluster
 - Perform a manual switchover (promote a standby) or a restart of a node
 - Issue TLS certificates for client authentication
 - Declare start and stop of a Kubernetes node maintenance
 - Destroy a cluster and all its PVC
 - Fence a cluster or a set of the instances
 - Hibernate a cluster
 - Generate jobs for benchmarking via pgbench and fio
 - Issue a new backup
 - Start pgadmin



Name: cluster-example
Namespace: default
System ID: 7100921006673293335
PostgreSQL Image: ghcr.io/cloudnative-pg/postgresql:14.3
Primary instance: cluster-example-2
Status: Cluster in healthy state
Instances: 3
Ready instances: 3
Current Write LSN: 0/C000060 (Timeline: 4 - WAL File: 00000004000000000000000C)

Certificates Status

Certificate Name	Expiration Date	Days Left Until Expiration
cluster-example-replication	2022-08-21 13:15:00 +0000 UTC	89.95
cluster-example-server	2022-08-21 13:15:00 +0000 UTC	89.95
cluster-example-ca	2022-08-21 13:15:00 +0000 UTC	89.95

Continuous Backup status

First Point of Recoverability: 2022-05-23T13:37:08Z
Working WAL archiving: OK
WALs waiting to be archived: 0
Last Archived WAL: 00000004000000000000000B @ 2022-05-23T13:42:09.37537Z
Last Failed WAL: -

Streaming Replication status

Name	Sent LSN	Write LSN	Flush LSN	Replay LSN	Write Lag	Flush Lag	Replay Lag	State	Sync State	Sync Priority
cluster-example-3	0/C000060	0/C000060	0/C000060	0/C000060	00:00:00	00:00:00	00:00:00	streaming	async	0
cluster-example-1	0/C000060	0/C000060	0/C000060	0/C000060	00:00:00	00:00:00	00:00:00	streaming	async	0

Instances status

Name	Database Size	Current LSN	Replication role	Status	QoS	Manager Version
cluster-example-3	33 MB	0/C000060	Standby (async)	OK	BestEffort	1.15.0
cluster-example-2	33 MB	0/C000060	Primary	OK	BestEffort	1.15.0
cluster-example-1	33 MB	0/C000060	Standby (async)	OK	BestEffort	1.15.0



NOT NEEDED DURING WORKSHOP
For illustrative purposes.

Install CNPG plugin

- In the web terminal run the script 01_install_plugin.sh:
`./01_install_plugin.sh`



Call to action: Check CNPG plugin

- Call the help for the CNPG Plugin, run:
`kubectl-cnp help`



Use case

Operator installation



NOT NEEDED DURING WORKSHOP

For illustrative purposes.

Operator Installation demonstration

- Install the operator
- Check the installed CNP Operator in the console
- Discover the features of the Operator in the OpenShift environment
- Check the installed CNP Operator in the web terminal



NOT NEEDED DURING WORKSHOP

For illustrative purposes.

Install the CNPG Operator and check the it in the web terminal

- In the web terminal install the operator:

```
./02_install_operator.sh
```

-> will require admin privs on Openshift



Call to action: Check the it in the web terminal

- In the web terminal check the installation of the operator:

```
./03_check_operator_installed.sh
```



Call to action: Check the installed CNPG Operator in the Openshift console

- In the OpenShift console navigate to:
 - -> Operators
 - -> Installed Operators
 - -> Klick on the Operator installed:

The screenshot shows the Red Hat OpenShift console interface. On the left is a sidebar with navigation links: Home, Favorites, Operators (expanded), OperatorHub, Installed Operators (selected), Helm, Workloads, Networking, Storage, Builds, and Observe. The main content area is titled 'Installed Operators' and shows a table of installed operators. A green arrow points to the 'EDB Postgres for Kubernetes' operator.

Name	Namespace	Managed Namespaces	Status	Last updated	Provided APIs
 cert-manager Operator for Red Hat OpenShift 1.17.0 provided by Red Hat	 cert-manager-operator	 cert-manager-operator	✓ Succeeded Up to date	14. Okt. 2025, 12:19	CertificateRequest Certificate Challenge ClusterIssuer View 4 more...
 EDB Postgres for Kubernetes 1.27.0 provided by EDB	 openshift-operators	All Namespaces	✓ Succeeded Up to date	15. Okt. 2025, 05:43	Backups Cluster Image Catalog Cluster Postgres Database View 6 more...

Call to action: Discover the features of the Operator in the OpenShift environment

Project: openshift-operators ▾

[Installed Operators](#) > Operator details



EDB Postgres for Kubernetes

1.27.0 provided by EDB



Details

[YAML](#)

[Subscription](#)

[Events](#)

[All instances](#)

[Backups](#)

[Cluster Image Catalog](#)

[Cluster](#)

[Postgres Database](#)

[Failover Qu](#)

Provided APIs

Backups

PostgreSQL backup (physical base backup)

[+ Create instance](#)

Cluster Image Catalog

A cluster-wide catalog of PostgreSQL operand images

[+ Create instance](#)

Cluster

PostgreSQL cluster (primary/standby architecture)

[+ Create instance](#)

Postgres Database

Declarative creation and management of a database on a Cluster

[+ Create instance](#)

Failover Quorum

FailoverQuorum contains the information about the current failover quorum status of a PG cluster

[+ Create instance](#)

Image Catalog

A catalog of PostgreSQL operand images

[+ Create instance](#)

Pooler

Postgres Publication

Scheduled Backups



Use case

Create the postgres cluster



Synchronizing the state of a Postgres database

- Being a DBMS, PostgreSQL is a stateful workload in Kubernetes
- Stateless workloads achieve HA and DR mainly through traffic redirection
- Stateful workloads require the state to be replicated in multiple locations:
 - **Storage-level** replication
 - **Application-level** replication (in our case, application = Postgres)
- Postgres has a very robust and powerful native replication system
 - We've built it
 - Founded on the Write Ahead Log
 - Read-only standby servers
 - Supports also synchronous replication controlled at the transaction level
- **We recommend application-level** over storage-level replication for Postgres



Bootstrap - different ways of creating a cluster

- Create a new cluster from scratch
 - “initdb”: named after the standard “initdb” process in PostgreSQL that initializes an instance
- Create a new cluster from an existing one:
 - Directly (“pg_basebackup”), using physical streaming replication
 - Directly (logical backup/restore) using pg_dump and pg_restore
 - Indirectly (“recovery”), from an object store
 - To the end of the WAL
 - Can be used to start independent replica clusters in continuous recovery
 - Using PITR



Storage management

- Storage is the most critical component for a database
- Direct support for Persistent Volume Claims (PVC)
 - We deliberately do not use Statefulsets
- The PVC storing the PGDATA is central to CloudNativePG
 - Our motto is: “PGDATA is worth a 1000 pods”
- Storage agnostic
- Freedom of choice
 - Local storage
 - Network storage
- Automated generation of PVC
- Support for PVC templates
 - Storage classes



Call to action: Configure and Install the Postgres cluster

- Prepare for cluster-creation (ensure minio secrets are in place)

`./04-prepare-cluster.sh`

- Create a new 3-node cluster by running

`./05_install_cluster.sh`

- Check the status of the cluster (using the CNP plugin):

`./06_show_status.sh`



Call to action: Create table test with 1000 rows

- Once cluster is running ... (minimum the primary) run the script:

```
./07_insert_data.sh
```

- Check data in the database, use the kubectl plugin to connect to the database:

```
echo "select count(*) from test;" | kubectl-cnp psql cluster-user<X>
```



Use case

Promote & Upgrade the postgres cluster



Rolling updates

- Update of a deployment with ~zero downtime
 - Standby servers are updated first
 - Then the primary:
 - supervised / unsupervised
 - switchover / restart
- When they are triggered:
 - Security update of Postgres images
 - Minor update of PostgreSQL
 - Configuration changes when restart is required
 - Update of the operator
 - Unless in-place upgrade is enabled



Call to action: Check the cluster status

- In terminal **1**: (prepare a terminal for status - and one to run the admin-commands):
 - Run the command
`./06_show_status.sh`
 - Review the output:
 - check Postgres version: "PostgreSQL Image: quay.io/enterprisedb/postgresql:**16.2**"
 - check "Continuous Backup status": **"Not configured"**
 - Check the updated cluster configuration - file cluster-example-upgrade.yaml
`less ./yaml/cluster-sample-upgrade.yaml`
 - Check Postgres version: "imageName: quay.io/enterprisedb/postgresql:**16.4**"
 - Check the Backup section



Call to action: Run the Promote and Upgrade

- With this step we will:
 - Promote node-2 to become the primary
 - Run the postgres minor update from the version 16.2 to 16.4
 - We will configure the WAL files backup to the S3 storage
- In the web terminal **2**:
 - Check the upgrade status:
`./06_show_status.sh`
- In the terminal **1**:
 - Run the script:
`./08_promote.sh`
 - Run the script:
`./09_upgrade.sh`



Use case Backup & Restore



Backup and Recovery - Part 1

- Continuous physical backup on “backup object stores”
 - Scheduled and on-demand base backups
 - Continuous WAL archiving (including parallel)
 - From primary or a standby
 - Support for recovery window retention policies (e.g. 30 days)
- Recovery means creating a new cluster starting from a “recovery object store”
 - Then pull WAL files (including in parallel) and replay them
 - Full (End of the WAL) or PITR
- Both rely on Barman Cloud technology
 - AWS S3
 - Azure Storage compatible
 - Google Cloud Storage
 - MinIO



Backup and Recovery - Part 2

- WAL management
 - Object store
- Physical Base backups
 - Object store
 - Kubernetes level backup integration (Velero/OADP, Veem Kasten K10, generic interface)
 - Kubernetes Volume Snapshots



Kubernetes Volume Snapshot: major advantages

- Transparent support for:
 - Incremental backup and recovery at block level
 - Differential backup and recovery at block level
 - Based on copy on write
- Leverage the storage class to manage the snapshots, including:
 - Data mobility across network (availability zones, Kubernetes clusters, regions)
 - Relay files on a secondary location in a different region, or any subsequent one
 - Encryption
- Enhances Very Large Databases (VLDB) adoption



Backup & Recovery via Snapshots: some numbers

Let's now talk about some initial benchmarks I have performed on volume snapshots using 3 `r5.4xlarge` nodes on AWS EKS with the `gp3` storage class. I have defined 4 different database size categories (tiny, small, medium, and large), as follows:

Cluster name	Database size	pgbench init scale	PGDATA volume size	WAL volume size	pgbench init duration
<i>tiny</i>	4.5 GB	300	8 GB	1 GB	67s
<i>small</i>	44 GB	3,000	80 GB	10 GB	10m 50s
<i>medium</i>	438 GB	3,0000	800 GB	100 GB	3h 15m 34s
<i>large</i>	4,381 GB	300,000	8,000 GB	200 GB	32h 47m 47s

The table below shows the results of both backup and recovery for each of them.

Cluster name	1st backup duration	2nd backup duration after 1hr of pgbench	Full recovery time
<i>tiny</i>	2m 43s	4m 16s	31s
<i>small</i>	20m 38s	16m 45s	27s
<i>medium</i>	2h 42m	2h 34m	48s
<i>large</i>	3h 54m 6s	2h 3s	2m 2s

<https://www.enterprisedb.com/postgresql-disaster-recovery-with-kubernetes-volume-snapshots-using-cloudnativepg>



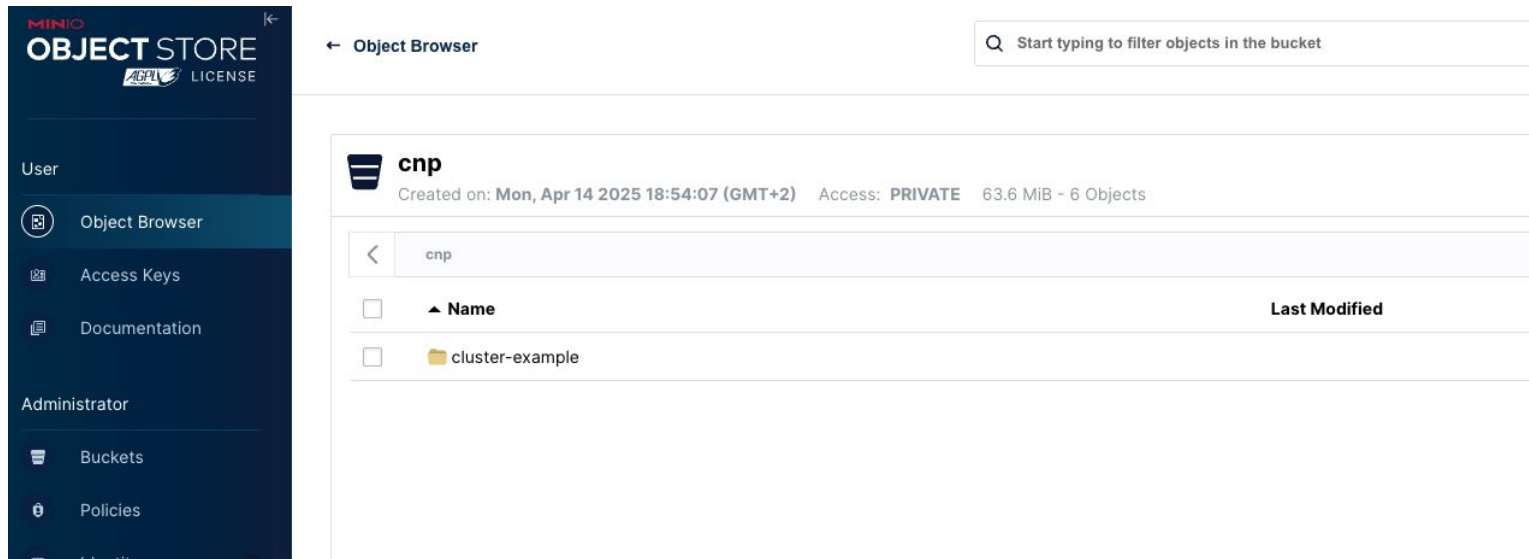
Call to action: Create the full backup

- With this step we will:
 - Create the full backup of the postgres cluster in the MinIO storage:
- In the web terminal 1:
 - Run the script:
`./10_backup_cluster.sh`
 - Check the backup status:
`./11_backup_describe.sh`



Call to action: Check Backup in MinIO UI

- Obtain the MinIO URL from the Slide **#56** and open the URL:
- Connect as user **minio** with the password: **edb-workshop**
- The page will appear:



The screenshot displays the MinIO Object Store interface. On the left is a dark blue sidebar with the 'MINIO OBJECT STORE' logo and 'AGPL LICENSE' text. The sidebar contains sections for 'User' (with 'Object Browser' selected), 'Access Keys', 'Documentation', and 'Administrator' (with 'Buckets' and 'Policies' listed). The main content area is titled 'Object Browser' and features a search bar with the placeholder text 'Start typing to filter objects in the bucket'. Below the search bar, a bucket named 'cnp' is shown with details: 'Created on: Mon, Apr 14 2025 18:54:07 (GMT+2)', 'Access: PRIVATE', and '63.6 MiB - 6 Objects'. A table lists the objects in the bucket, with columns for a checkbox, 'Name', and 'Last Modified'. One object is listed: a folder icon followed by 'cluster-example'.

	Name	Last Modified
☐	cluster-example	



Call to action: Restore the database from the backup

- With this step we will:
 - Create the new cluster cluster-restore
 - Restore the full backup created in the previous step in the new cluster:
- In the terminal 1:
 - Run the restore:
`./12_restore_cluster.sh`
 - Check the creation status:
`kubectl get pods -w` # after creation stop the execution with <ctrl>+c
 - Check the table test in the cluster-restore, run the script:
`oc exec -it cluster-restore-user<X>-1 -- psql -U postgres -c "select count(*) from test;"`
 - Delete the cluster-restore-user<x> to avoid resource problems during the workshop:
`oc delete cluster cluster-restore-user<X>`



Use case: Failover



Call to action: Run failover test

- With this step we will:
 - Delete the primary database of the cluster cluster-example
 - Check the cluster status in the another terminal window
- In the web terminal 1:
 - Run the script:
`./13_failover.sh`
- In the web terminal **2**:
 - Check the failover cluster status:
`./06_show_status.sh`



Use case

Scale-out and scale-down



Scale up and down of replicas

- The operator allows you to scale up and down the number of instances in a PostgreSQL cluster.
- New replicas are started up from the primary server and participate in the cluster's HA infrastructure.
- The CRD declares a "scale" subresource that allows you to use the `kubectl scale` command.



Call to action: Scale-out the postgres cluster

- With this step we will:
 - Add the 1 standby to the cluster
- In the web terminal 1:
 - Run the script:
`./14_scale_out.sh` (using `-replicas=X...` another way would be to update the YAML)
- In the web terminal **2**:
 - Check the cluster status:
`./06_show_status.sh`



Call to action: Scale-down the postgres cluster

- With this step we will:
 - Remove 2 standby pods from the cluster
- In the web terminal 1:
 - Run the script:
`./15_scale_down.sh`
- In the web terminal **2**:
 - Check the cluster status:
`./06_show_status.sh`



Use Case Fencing



Fencing

- **Fencing** is the process of protecting the data in one, more, or even all instances of a PostgreSQL cluster when they appear to be malfunctioning.
- When an instance is fenced, the PostgreSQL server process is guaranteed to be shut down, while the pod is kept running.
- This ensures that, until the fence is lifted, data on the pod isn't modified by PostgreSQL and that you can investigate file system for debugging and troubleshooting purposes.



Call to action: Stop postgres process on the pod

- In the web terminal 1:
 - Run the script:

```
./30_fencing_on.sh
```

- In the web terminal **2**:
 - Check the cluster status:

```
./06_show_status.sh
```



Call to action: Start the postgres process on the pod

- In the terminal 1:
 - Run the script:
`./31_fencing_off.sh`
- In the terminal **2**:
 - Check the cluster status:
`./06_show_status.sh`



Use case Hibernation



Hibernation

- CloudNativePG supports **hibernation** of a running PostgreSQL cluster in a declarative manner
 - through the `cnpg.io/hibernation` annotation
- Hibernation enables saving CPU power by removing the database pods while keeping the database PVCs
- This feature simulates scaling to 0 instances.



Call to action: Stop the postgres cluster

- In the terminal 1:
 - Run the script:

```
./32_hibernation_on.sh
```

- In the terminal **2**:
 - Check the cluster status:

```
./06_show_status.sh
```



Call to action: Start the postgres cluster

- In the terminal 1:
 - Run the script:

```
./33_hibernation_off.sh
```

- In the terminal **2**:
 - Check the cluster status:

```
./06_show_status.sh
```



Use case

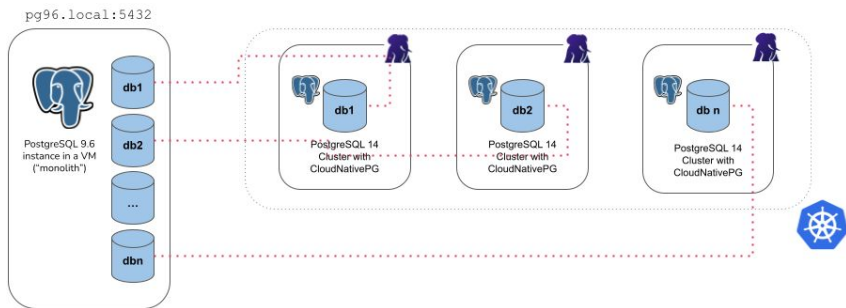
Database Migration / Major Version Upgrade



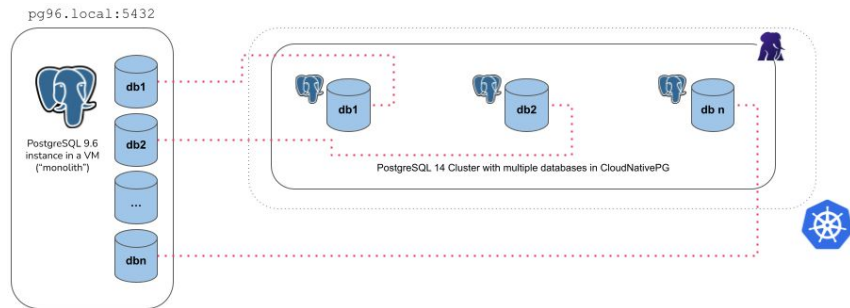
Database Migration

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting “**import**” in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:

Microservices:



Monolith:



Call to action: Run migration or out of the place upgrade

- In the web terminal 1:

- Run the script:

- ```
./20_upgrade_major_version.sh
```

- Check the cluster creation process:

- ```
kubectl get pods -w
```

- Check the table test in the cluster-user<X>-17, run the command:

- Connect to the cluster:

- ```
oc cnp psql cluster-user<X>-17
```

- Connect to the database app:

- ```
\c app
```

- Run sql commands:

- ```
select version();
```

- ```
select count(*) from test;
```



What more?
(some additional features from EDB)

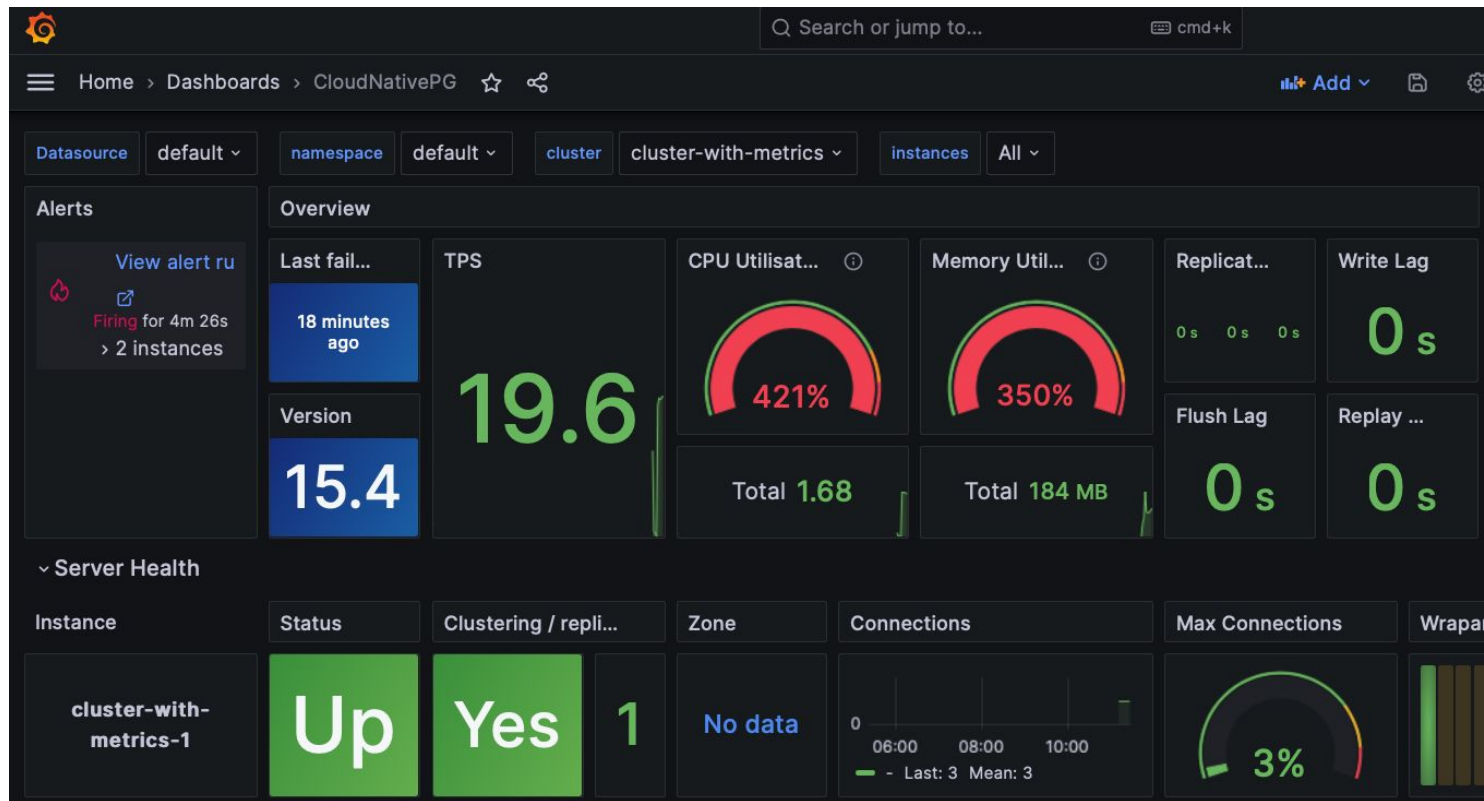


What we didn't show you today

- PgBouncer (Pooler) integration
 - Create a PgBouncer deployment and automatically configure to the cluster.
- Monitoring using Prometheus and Grafana
 - Exporting to OpenMetrics (Prometheus)



Grafana Dashboard



Advanced Security



Password policy management

DBA managed password profiles, compatible with Oracle profiles



Audit compliance

Track and analyze database activities and user connections



Virtual private databases

Fine grained access control limits user views



EDB/SQL protect

SQL firewall, screens queries for common attack profiles



Data redaction

Protect sensitive information for GDPR, PCI and HIPAA compliance



Code protection

Protects sensitive IP, algorithms or financial policies

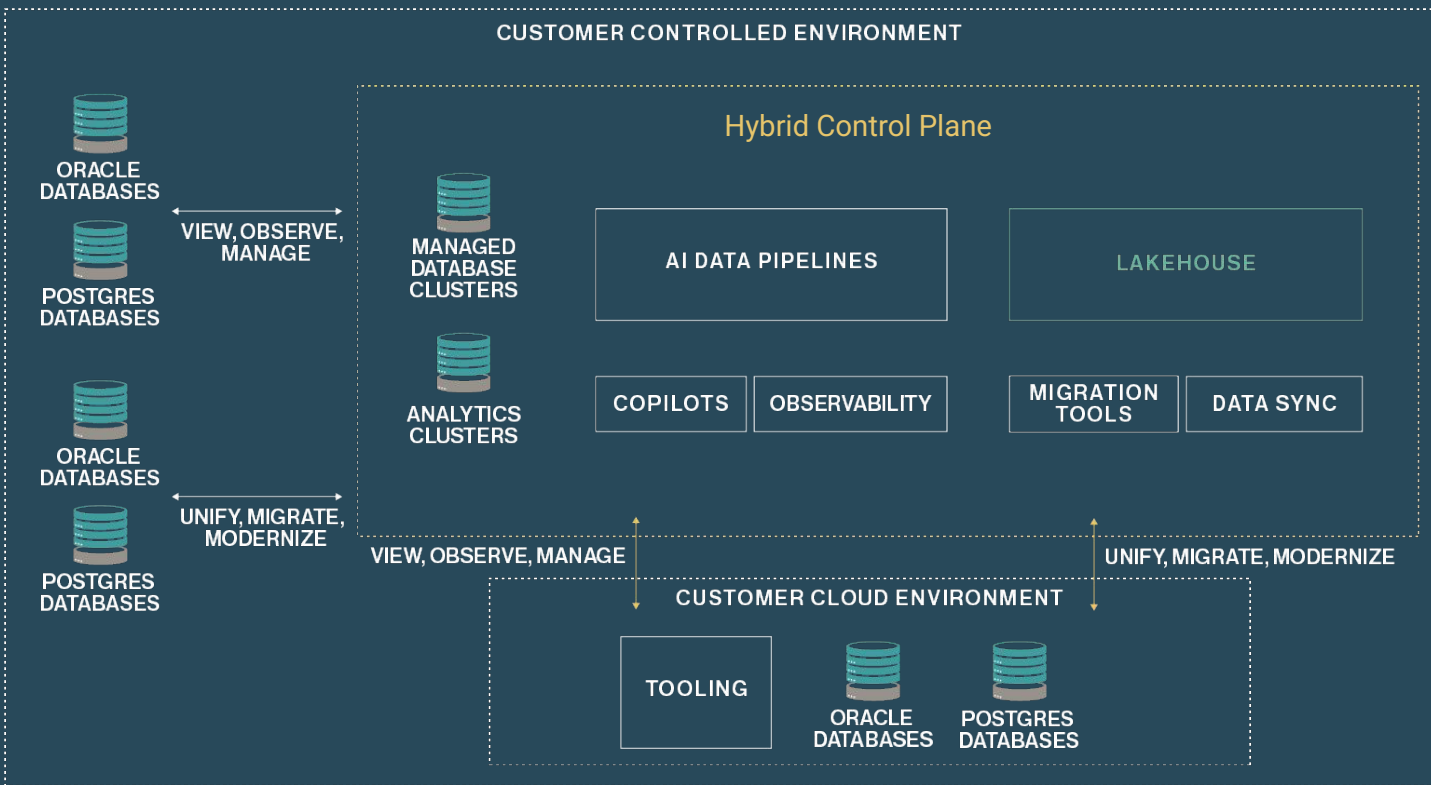


Transparent Data Encryption (EDB-only features)

- Transparent Data Encryption (TDE) is a feature of EDB Postgres Advanced Server and EDB Postgres Extended Server that prevents unauthorized viewing of data in operating system files on the database server and on backup storage
- Data encryption and decryption is managed by the database and does not require application changes or updated client drivers
- EDB Postgres Advanced Server and EDB Postgres Extended Server provide hooks to key management that is external to the database allowing for simple passphrase encrypt/decrypt or integration with enterprise key management solutions, with initial support for:
 - Amazon AWS Key Management Service (KMS)
 - Google Cloud - Cloud Key Management Service
 - Microsoft Azure Key Vault
 - HashiCorp Vault (KMIP Secrets Engine and Transit Secrets Engine)
 - Thales CipherTrust Manager
- Data will be unintelligible for unauthorized users if stolen or misplaced



Hybrid Control Plane at a glance



Hybrid Control Plane

LIVE DEMO

